Interprocess Communication

A Taxonomy of IPC Facilities

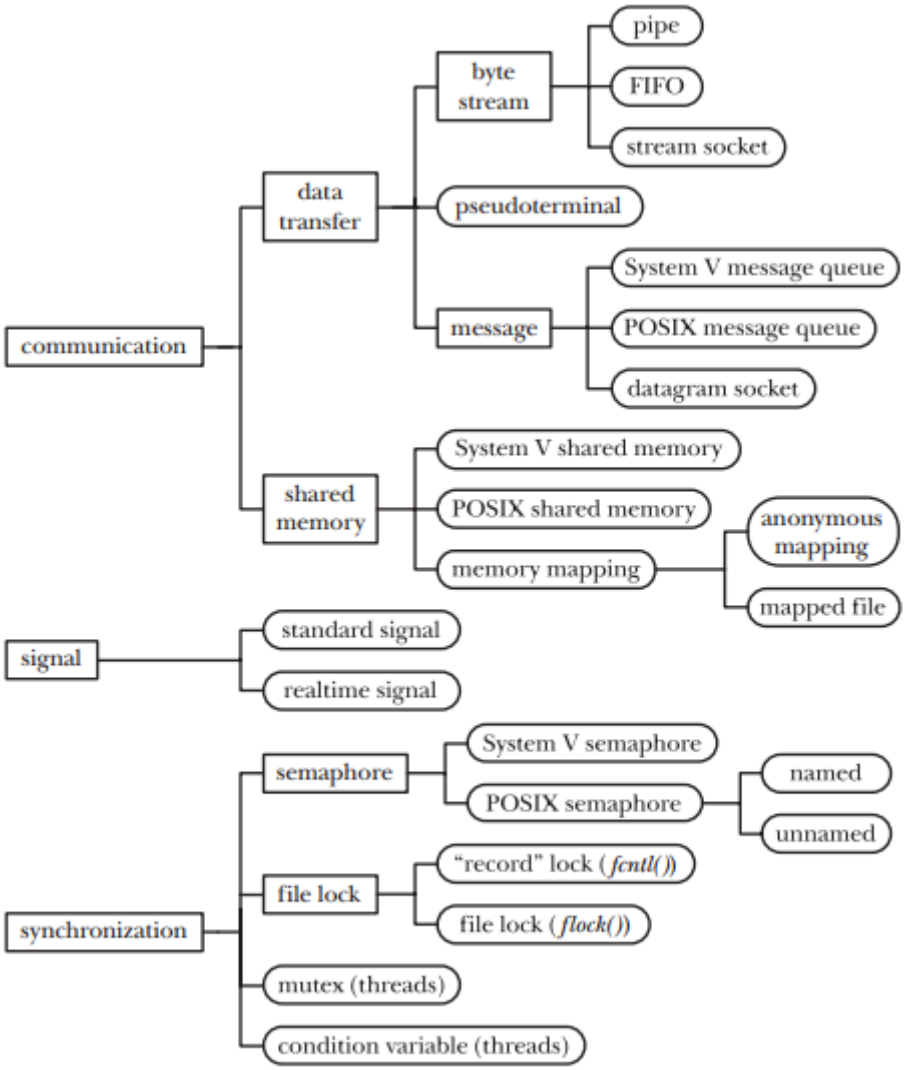


Figure 43-1: A taxonomy of UNIX IPC facilities

Figure 43-1 summarizes the rich variety of UNIX communication and synchronization facilities, dividing them into three broad functional categories:

- ***Communication:*** These facilities are concerned with exchanging data between processes.
- ***Synchronization:*** These facilities are concerned with synchronizing the actions of processes or threads.

- ***Signals:*** Although signals are intended primarily for other purposes, they can be used as a synchronization technique in certain circumstances. More rarely, signals can be used as a communication technique: the signal number itself is a form of information, and real time signals can be accompanied by associated data (an integer or a pointer).

Communication Facilities

The various communication facilities shown in Figure 43-1 allow processes to exchange data with one another. (These facilities can also be used to exchange data between the threads of a single process, but this is seldom necessary, since threads can exchange information via shared global variables.)

Data-transfer facilities: The key factor distinguishing these facilities is the notion of writing and reading. In order to communicate, one process writes data to the IPC facility, and another process reads the data. These facilities require two data transfers between user memory and kernel memory: one transfer from user memory to kernel memory during writing, and another transfer from kernel memory to user memory during reading. (Figure 43-2 shows this situation for a pipe.)

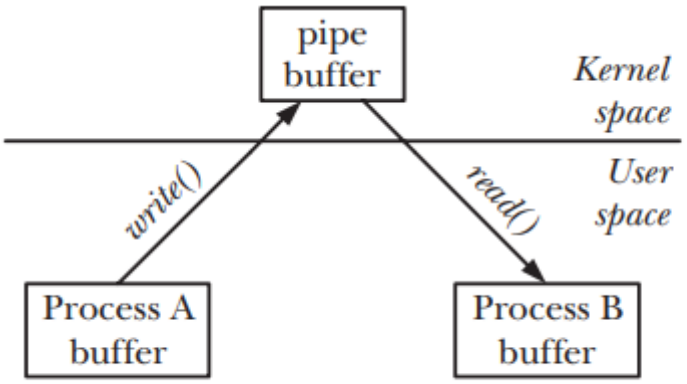


Figure 43-2: Exchanging data between two processes using a pipe

Shared memory: Shared memory allows processes to exchange information by placing it in a region of memory that is shared between the processes. (The kernel accomplishes this by making page-table entries in each process point to the same pages of RAM, as shown in Figure 49-2, on page 1026.) A process can make data available to other processes by placing it in the shared memory region. Because communication doesn't require system calls or data transfer between user memory and kernel memory, shared memory can provide very fast communication.

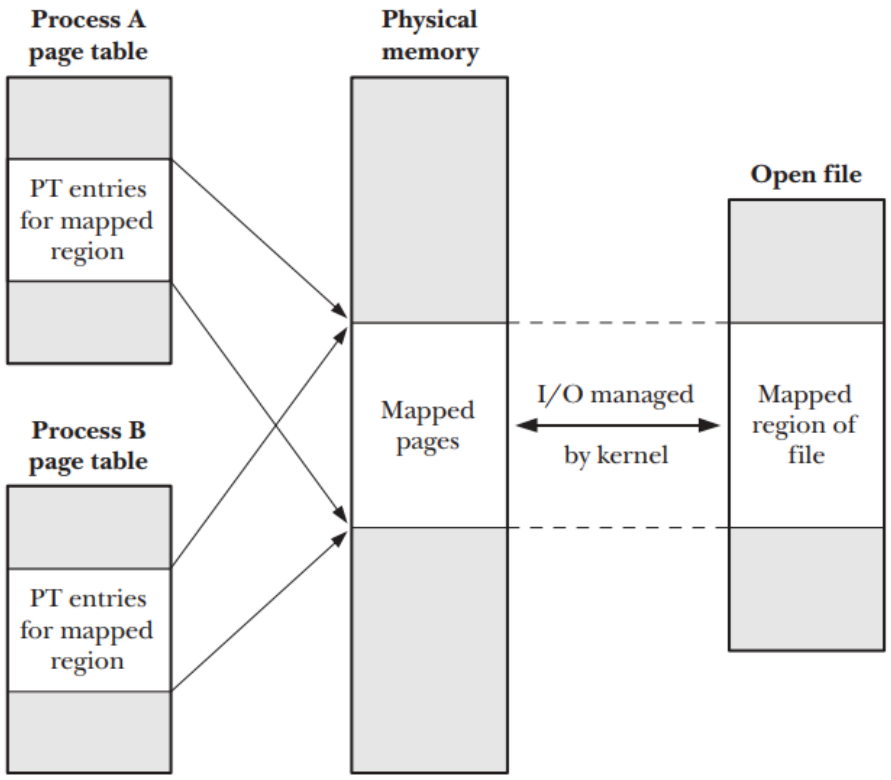


Figure 49-2: Two processes with a shared mapping of the same region of a file

Data transfer

We can further break data-transfer facilities into the following subcategories:

Byte stream: The data exchanged via pipes, FIFOs, and datagram sockets is an undelimited byte stream. Each read operation may read an arbitrary number of bytes from the IPC facility, regardless of the size of blocks written by the writer.

Message: The data exchanged via System V message queues, POSIX message queues, and datagram sockets takes the form of delimited messages. Each read operation reads a whole message, as written by the writer process. It is not possible to read part of a message, leaving the remainder on the IPC facility; nor is it possible to read multiple messages in a single read operation.

Pseudoterminals: A pseudoterminal is a communication facility intended for use in specialized situations. We will provide details later.

A few general features distinguish data-transfer facilities from shared memory:

- Although a data-transfer facility may have multiple readers, reads are destructive. A read operation consumes data, and that data is not available to any other process.

- Synchronization between the reader and writer processes is automatic. If a reader attempts to fetch data from a data-transfer facility that currently has no data, then (by default) the read operation will block until some process writes data to the facility.

Shared memory

Most modern UNIX systems provide three flavors of shared memory: System V shared memory, POSIX shared memory, and memory mappings. We consider the differences between them when describing the facilities in later chapters.

Note the following general points about shared memory:

- Although shared memory provides fast communication, this speed advantage is offset by the need to synchronize operations on the shared memory. For example, one process should not attempt to access a data structure in the shared memory while another process is updating it. A semaphore is the usual synchronization method used with shared memory.

- Data placed in shared memory is visible to all of the processes that share that memory. (This contrasts with the destructive read semantics described above for data-transfer facilities.)

Synchronization Facilities

The synchronization facilities shown in Figure 43-1 allow processes to coordinate their actions. Synchronization allows processes to avoid doing things such as simultaneously updating a shared memory region or the same part of a file. Without synchronization, such simultaneous updates could cause an application to produce incorrect results.

Semaphores: A semaphore is a kernel-maintained integer whose value is never permitted to fall below 0. A process can decrease or increase the value of a semaphore. If an attempt is made to decrease the value of the semaphore below 0, then the kernel blocks the operation until the semaphore's value increases to a level that permits the operation to be performed. (Alternatively, the process can request a non blocking operation; then, instead of blocking, the kernel causes the operation to return immediately with an error indicating that the operation can't be performed immediately.)

File locks: File locks are a synchronization method explicitly designed to coordinate the actions of multiple processes operating on the same file. They can also be used to coordinate access to other shared resources. File locks come in two flavors: read (shared) locks and write (exclusive) locks. Any number of processes can hold a read

lock on the same file (or region of a file). However, when one process holds a write lock on a file (or file region), other processes are prevented from holding either read or write locks on that file (or file region). Linux provides file-locking facilities via the `flock()` and `fcntl()` system calls. The `flock()` system call provides a simple locking mechanism, allowing processes to place a shared or an exclusive lock on an entire file. Because of its limited functionality, `flock()` locking facility is rarely used nowadays. The `fcntl()` system call provides record locking, allowing processes to place multiple read and write locks on different regions of the same file.

Mutexes and condition variables: These synchronization facilities are normally used with POSIX threads

Pipes and FIFOs

Pipes are the oldest method of IPC on the UNIX system.
Pipes provide an elegant solution to a frequent requirement: having created two processes to run different programs (commands), how can the shell allow the output produced by one process to be used as the input to the other process? Pipes can be used to pass data between related processes (the meaning of related will become clear later). FIFOs are a variation on the pipe concept. The important difference is that FIFOs can be used for communication between any processes.

Overview

Every user of the shell is familiar with the use of pipes in commands such as the following, which counts the number of files in a directory:

```
$ ls | wc -l
```

In order to execute the above command, the shell creates two processes, executing ls and wc, respectively. (This is done using fork() and exec(), which are described in Chapters 24 and 27.) Figure 44-1 shows how the two processes employ the pipe.

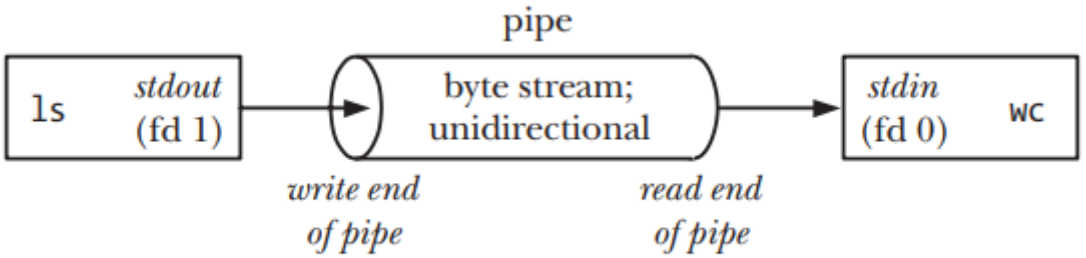


Figure 44-1: Using a pipe to connect two processes

One point to note in Figure 44-1 is that the two processes are connected to the pipe so that the writing process (ls) has its standard output (file descriptor 1) joined to the write end of the pipe, while the reading process (wc) has its standard input (file descriptor 0) joined to the read end of the pipe. In effect, these two processes are unaware of the existence of the pipe; they just read from and write to the standard file descriptors. The shell must do some work in order to set things up in this way

A pipe is a byte stream

When we say that a pipe is a byte stream, we mean that there is no concept of messages or message boundaries when using a pipe. The process reading from a pipe can read blocks of data of any size, regardless of the size of blocks written by the writing process. Furthermore, the data passes through the pipe sequentially—

bytes are read from a pipe in exactly the order they were written. It is not possible to randomly access the data in a pipe using `lseek()`.

Reading from a pipe

Attempts to read from a pipe that is currently empty block until at least one byte has been written to the pipe. If the write end of a pipe is closed, then a process reading from the pipe will see end-of-file (i.e., `read()` returns 0) once it has read all remaining data in the pipe.

Pipes are unidirectional

Data can travel only in one direction through a pipe. One end of the pipe is used for writing, and the other end is used for reading.

Writes of up to PIPE_BUF bytes are guaranteed to be atomic

If multiple processes are writing to a single pipe, then it is guaranteed that their data won't be intermingled if they write no more than `PIPE_BUF` bytes at a time.

When writing blocks of data larger than `PIPE_BUF` bytes to a pipe, the kernel may transfer the data in multiple smaller pieces, appending further data as the reader removes bytes from the pipe. (The `write()` call blocks until all of the data has been written to the pipe.) When there is only a single process writing to a pipe (the usual case), this doesn't matter. However, if there are multiple writer processes, then writes of large blocks may be broken into segments of arbitrary size (which may be smaller than `PIPE_BUF` bytes) and interleaved with writes by other processes.

The `PIPE_BUF` limit affects exactly when data is transferred to the pipe. When writing up to `PIPE_BUF` bytes, `write()` will block if necessary until sufficient space is available in the pipe so that it can complete the operation atomically. When more than `PIPE_BUF` bytes are being written, `write()` transfers as much data as possible to fill the pipe, and then blocks until data has been removed from the pipe by some reading process. If such a blocked `write()` is interrupted by a signal handler, then the call unblocks and returns a count of the number of bytes successfully transferred, which will be less than was requested (a so-called partial write).

Pipes have a limited capacity

A pipe is simply a buffer maintained in kernel memory. This buffer has a maximum capacity. Once a pipe is full, further writes to the pipe block until the reader removes some data from the pipe.

Creating and Using Pipes

The pipe() system call creates a new pipe.

```
#include <unistd.h>

int pipe(int filedes[2]);
```

Returns 0 on success, or −1 on error

A successful call to pipe() returns two open file descriptors in the array filedes: one for the read end of the pipe (filedes[0]) and one for the write end (filedes[1]).

As with any file descriptor, we can use the read() and write() system calls to perform I/O on the pipe. Once written to the write end of a pipe, data is immediately available to be read from the read end. A read() from a pipe obtains the lesser of the number of bytes requested and the number of bytes currently available in the pipe (but blocks if the pipe is empty).

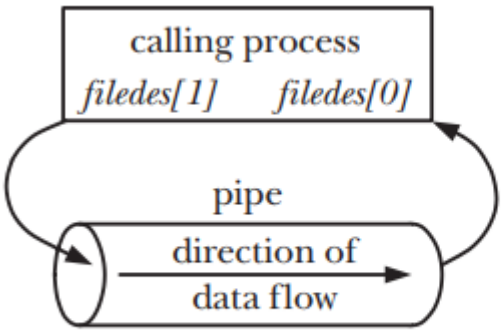
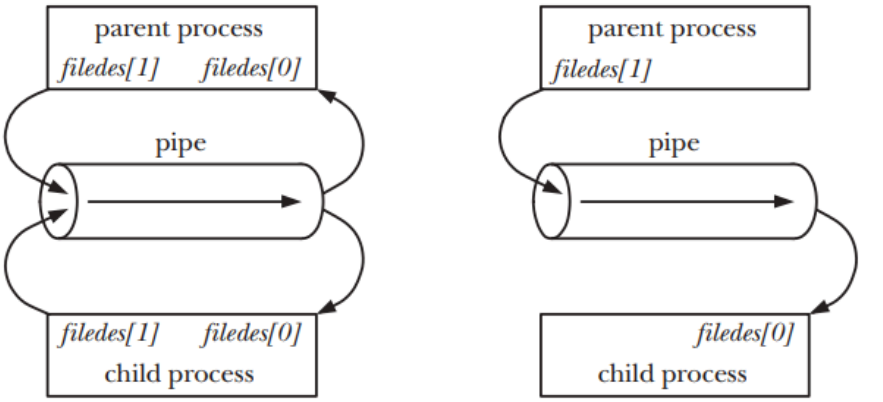


Figure 44-2: Process file descriptors after creating a pipe

A pipe has few uses within a single process (we consider one in Section 63.5.2). Normally, we use a pipe to allow communication between two processes. To connect two processes using a pipe, we follow the pipe() call with a call to fork(). During a fork(), the child process inherits copies of its parent's file descriptors (Section 24.2.1), bringing about the situation shown on the left side of Figure 44-3.



a) After *fork()* b) After closing unused descriptors

Figure 44-3: Setting up a pipe to transfer data from a parent to a child

While it is possible for the parent and child to both read from and write to the pipe, this is not usual. Therefore, immediately after the fork(), one process closes its descriptor for the write end of the pipe, and the other closes its descriptor for the read end. For example, if the parent is to send data to the child, then it would close its read descriptor for the pipe, filedes[0], while the child would close its write descriptor for the pipe, filedes[1], bringing about the situation shown on the right side of Figure 44-3. The code to create this setup is shown in Listing 44-1.

Listing 44-1: Steps in creating a pipe to transfer data from a parent to a child

```
int filedes[2];

if (pipe(filedes) == -1)                    /* Create the pipe */
    errExit("pipe");

switch (fork()) {                           /* Create a child process */
case -1:
    errExit("fork");

case 0:  /* Child */
    if (close(filedes[1]) == -1)            /* Close unused write end */
        errExit("close");

    /* Child now reads from pipe */
    break;

default: /* Parent */
    if (close(filedes[0]) == -1)            /* Close unused read end */
        errExit("close");

    /* Parent now writes to pipe */
    break;
}
```

Pipes allow communication between related processes

In the discussion so far, we have talked about using pipes for communication between a parent and a child process. However, pipes can be used for communication between any two (or more) related processes, as long as the pipe was created by a common ancestor before the series of fork() calls that led to the existence of the processes. (This is what we meant when we referred to related processes at the beginning of this chapter.) For example, a pipe could be used for communication between a process and its grandchild. The first process creates the pipe, and then forks a child that in turn forks to yield the grandchild. A common scenario is that a pipe is used for communication between two siblings—their parent creates the pipe, and then creates the two children. This is what the shell does when building a pipeline.

Example program

The program in Listing 44-2 demonstrates the use of a pipe for communication between parent and child processes. This example demonstrates the byte-stream nature of pipes referred to earlier—the parent writes its data in a single operation, while the child reads data from the pipe in small blocks.

The main program calls pipe() to create a pipe, and then calls fork() to create a child. After the fork(), the parent process closes its file descriptor for the read end of the pipe, and writes the string given as the program's command-line argument to the write end of the pipe. The parent then closes the rear end of the pipe, and calls wait() to wait for the child to terminate. After closing its file descriptor for the write end of the pipe, the child process enters a loop where it reads blocks of data (of up to BUF_SIZE bytes) from the pipe and writes them to standard output. When the child encounters end-of-file on the pipe, it exits the loop, writes a trailing newline character, closes its descriptor for the read end of the pipe, and terminates.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>
#include <stdarg.h>

#define BUF_SIZE 10

void errExit(const char *format, ...)
{
    va_list argList;
```

```
    va_start(argList, format);
    vfprintf(stderr, format, argList);  // Print formatted error message
    va_end(argList);
    exit(EXIT_FAILURE);  // Exit program with failure status
}

int main(int argc, char *argv[])
{
    int pfd[2]; /* Pipe file descriptors */
    char buf[BUF_SIZE];
    ssize_t numRead;

    if (argc != 2 || strcmp(argv[1], "--help") == 0)
        errExit("%s string\n", argv[0]);  // Print usage error if arguments
are invalid

    if (pipe(pfd) == -1)  /* Create the pipe */
        errExit("pipe");

    switch (fork())
    {
    case -1:
        errExit("fork");

    case 0:  /* Child - reads from pipe */
        if (close(pfd[1]) == -1)  /* Write end is unused */
            errExit("close - child");

        for (;;)
        { /* Read data from pipe, echo on stdout */
            numRead = read(pfd[0], buf, BUF_SIZE);
            if (numRead == -1)
                errExit("read");
            if (numRead == 0)
                break;  /* End-of-file */
            if (write(STDOUT_FILENO, buf, numRead) != numRead)
                errExit("child - partial/failed write");
        }
        write(STDOUT_FILENO, "\n", 1);  // Print newline
        if (close(pfd[0]) == -1)
            errExit("close - child");
        exit(EXIT_SUCCESS);  // Exit child process
```

```
    default:  /* Parent - writes to pipe */
        if (close(pfd[0]) == -1)  /* Read end is unused */
            errExit("close - parent");

        if (write(pfd[1], argv[1], strlen(argv[1])) != strlen(argv[1]))
            errExit("parent - partial/failed write");

        if (close(pfd[1]) == -1)  /* Child will see EOF */
            errExit("close - parent");

        wait(NULL);  /* Wait for child to finish */
        exit(EXIT_SUCCESS);  // Exit parent process
    }
}
```

Here's an example of what we might see when running the program in Listing 44-2:

```
$ ./simple_pipe 'It was a bright cold day in April, '\
'and the clocks were striking thirteen.'
It was a bright cold day in April, and the clocks were striking thirteen.
```

Pipes as a Method of Process Synchronization

In Section 24.5, we looked at how signals could be used to synchronize the actions of parent and child processes in order to avoid race conditions. Pipes can be used to achieve a similar result, as shown by the skeleton program in Listing 44-3. This program creates multiple child processes (one for each command-line argument), each of which is intended to accomplish some action, simulated in the example program by sleeping for some time. The parent waits until all children have completed their actions.

To perform the synchronization, the parent builds a pipe before creating the child processes. Each child inherits a file descriptor for the write end of the pipe and closes this descriptor once it has completed its action. After all of the children have closed their file descriptors for the write end of the pipe, the parent's read() from the pipe will complete, returning end-of-file (0). At this point, the parent is free to carry on to do other work. (Note that closing the unused write end of the pipe in the parent r is essential to the correct operation of this technique; otherwise, the parent would block forever when trying to read from the pipe.)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
```

```c
#include <sys/wait.h>
#include <string.h>
#include <errno.h>
#include <stdarg.h>

#define BUF_SIZE 10

void errExit(const char *format, ...)
{
    va_list argList;
    va_start(argList, format);
    vfprintf(stderr, format, argList);  // Print formatted error message
    va_end(argList);
    exit(EXIT_FAILURE);  // Exit program with failure status
}

int main(int argc, char *argv[]) {
    int pfd[2]; /* Process synchronization pipe */
    int j, dummy;

    if (argc < 2 || strcmp(argv[1], "--help") == 0)
        errExit("%s sleep-time...\n", argv[0]);

    setbuf(stdout, NULL); /* Make stdout unbuffered */

    printf("Parent started\n");

    if (pipe(pfd) == -1)
        errExit("pipe");

    for (j = 1; j < argc; j++) {
        switch (fork()) {
            case -1:
                errExit("fork");

            case 0: /* Child */
                if (close(pfd[0]) == -1) /* Close unused read end */
                    errExit("close");

                /* Simulate processing */
                sleep(atoi(argv[j]));
```

```
                printf("Child %d (PID=%ld) closing pipe\n", j,
(long)getpid());

                if (close(pfd[1]) == -1)
                    errExit("close");

                _exit(EXIT_SUCCESS);

            default: /* Parent loops to create next child */
                break;
        }
    }

    /* Parent: Close write end of pipe to detect EOF */
    if (close(pfd[1]) == -1)
        errExit("close");

    /* Parent waits for EOF */
    if (read(pfd[0], &dummy, 1) != 0)
        errExit("parent didn't get EOF");

    printf("Parent ready to go\n");

    exit(EXIT_SUCCESS);
}
```

The following is an example of what we see when we use the program in Listing 44-3 to create three children that sleep for 4, 2, and 6 seconds:

```
$ ./pipe_sync 4 2 6
08:22:16  Parent started
08:22:18  Child 2 (PID=2445) closing pipe
08:22:20  Child 1 (PID=2444) closing pipe
08:22:22  Child 3 (PID=2446) closing pipe
08:22:22  Parent ready to go
```

Using Pipes to Connect Filters

When a pipe is created, the file descriptors used for the two ends of the pipe are the next lowest-numbered descriptors available. Since, in normal circumstances, descriptors 0, 1, and 2 are already in use for a process, some higher-numbered descriptors will be allocated for the pipe. So how do we bring about the situation shown in Figure 44-1, where two filters

(i.e., programs that read from stdin and write to stdout) are connected using a pipe, such that the standard output of one program is directed into the pipe and the standard input of the other is taken from the pipe? And in particular, how can we do this without modifying the code of the filters themselves?

Example program

The program in Listing 44-4 uses the techniques described in this section to bring about the setup shown in Figure 44-1. After building a pipe, this program creates two child processes. The first child binds its standard output to the write end of the pipe and then execs ls. The second child binds its standard input to the read end of the pipe and then execs wc.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void errExit(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int main() {
    int pfd[2]; // Pipe file descriptors

    if (pipe(pfd) == -1)
        errExit("pipe");

    // First child process (executes "ls")
    switch (fork()) {
        case -1:
            errExit("fork");
        case 0: // Child process
            close(pfd[0]); // Close unused read end

            if (pfd[1] != STDOUT_FILENO) {
                if (dup2(pfd[1], STDOUT_FILENO) == -1)
                    errExit("dup2 1");
                close(pfd[1]);
            }
            execlp("ls", "ls", NULL);
            errExit("execlp ls");
        default:
```

```
            break;
    }

    // Second child process (executes "wc -l")
    switch (fork()) {
        case -1:
            errExit("fork");
        case 0: // Child process
            close(pfd[1]); // Close unused write end

            if (pfd[0] != STDIN_FILENO) {
                if (dup2(pfd[0], STDIN_FILENO) == -1)
                    errExit("dup2 2");
                close(pfd[0]);
            }
            execlp("wc", "wc", "-l", NULL);
            errExit("execlp wc");
        default:
            break;
    }

    // Parent closes both pipe ends and waits for children
    close(pfd[0]);
    close(pfd[1]);

    wait(NULL);
    wait(NULL);

    return EXIT_SUCCESS;
}
```

Exercise

1. Simple Pipe Communication

📌 **Concepts:** `pipe()`, `fork()`, `write()`, `read()`
🔹 **Task:** Create a parent-child process where the parent writes a message to the pipe, and the child reads and prints it.

2. Multiple Children with a Pipe

📌 **Concepts:** `pipe()`, multiple `fork()` calls, inter-process communication
🔹 **Task:** Implement two child processes—one writes numbers (`1 2 3 4 5`) to the pipe, and the other reads and sums them.

3. Simulating `ls | grep` Using Pipes

📌 **Concepts:** `pipe()`, `dup2()`, `execlp()`
🔹 **Task:** Implement `ls | grep .c` using pipes, where one process executes `ls` and the other filters files ending with `.c`.

4. Creating a Two-Way Pipe Communication

📌 **Concepts:** `pipe()`, `fork()`, bidirectional communication
🔹 **Task:** Parent sends a number to the child, the child squares it and sends it back to the parent, which prints the result.

5. Implementing a Shell with Pipe Support

📌 **Concepts:** `fork()`, `execvp()`, `dup2()`, multiple pipes
🔹 **Task:** Build a shell that supports **single and multiple pipes**, e.g., `ls | grep .c | wc -l`.

6. Concurrent Processing Using Pipes

📌 **Concepts:** `fork()`, `wait()`, pipes for synchronization
🔹 **Task:** Create 3 child processes that each compute a different mathematical operation on a given input and send results back to the parent.

7. Redirecting `stdout` to a File Using `dup2()`

📌 **Concepts:** `dup2()`, file descriptors
🔹 **Task:** Write a program that redirects standard output to a file (`output.txt`) without modifying `printf()` statements.

8. Implementing `tee` Command Using Pipes

📌 **Concepts:** `pipe()`, `fork()`, `dup2()`, file redirection
🔹 **Task:** Simulate the `tee` command, which writes input to both the console and a file.

9. Parent-Child Synchronization Using Pipes

📌 **Concepts:** `pipe()`, `read()` blocking behavior, `wait()`
🔹 **Task:** Parent process waits for a signal from the child before continuing execution.

10. Multiple Writers to a Pipe

📌 **Concepts:** `pipe()`, `fork()`, race conditions
🔹 **Task:** Create multiple child processes that write data to the same pipe and observe interleaving behavior.